P4049R0
Relaxing and extending std::copy

Published Proposal, 2026-04-02

Author:
Giuseppe D'Angelo

Audience:
SG23, SG9, SG1, SG18, LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

The preconditions on the copy family of algorithms (`copy`, `move`, `copy_backward`, `move_backward`) exist in order to prevent overlapping writes from garbling the output. In this paper we argue that these preconditions are a source of "gratuitous" undefined behavior: they are incomplete, and they do not enable any useful optimization. Therefore we propose to remove them; and, for contiguous iterators, guarantee the correct result for all overlaps.

Table of Contents

§ 1. Changelog

- R0
  - First submission

§ 2. Motivation

The *copy* family of algorithms copy-assigns (or move-assigns) objects from a source range to a destination range.

All algorithms in this family impose preconditions on the relationship between the source and destination ranges. In general, **they do not allow for arbitrarily overlapping ranges**.

The exact preconditions depend on the algorithm:

Algorithm	Preconditions on the source/destination ranges	Notes
`copy`, `move` (sequential versions)	The beginning of the destination range must not be in the source range.	
`copy`, `move` (parallel versions)	The source and destination ranges must not overlap.	
`copy_backward`, `move_backward`	The end of the destination range must not be in the source range.	These algorithms do not have parallel versions.
`copy_n`	No preconditions.	This is Library Defect [LWG3089].
`copy_if`, `replace_copy`, `replace_copy_if`, `remove_copy`, `remove_copy_if`, `unique_copy`	The source range does not overlap with the part of the destination range which is written into.	Amended by [P3179R8] for C++26 to only take into account the elements that are actually copied. Before C++26, any overlap was forbidden.
`reverse_copy`, `rotate_copy`	The source and destination ranges must not overlap.	The parallel range versions take the destination's range size into account.

The preconditions listed above apply to all the versions of a given algorithm: the "traditional" version(s) in namespace `std`, the range-based one(s) in `std::ranges`, and the parallel one(s).

§ 2.1. What overlaps are allowed, and why?

Unlike most other algorithms in the family, `copy`, `move`, `copy_backward` and `move_backward` do allow for *some* overlap between source and destination. This is intentional, as these algorithms serve as building blocks for container operations that need to shift elements within the same storage.

For instance, for the `std::copy` algorithm, [alg.copy]/2 states:

> ```
> template<class InputIterator, class OutputIterator>
>   constexpr OutputIterator copy(InputIterator first, InputIterator last,
>                                 OutputIterator result);
> ```
>
> *Preconditions*: `result` is not in the range `[first, last)`.
>
> *Effects*: Copies elements in the range `[first, last)` into the range `[result, result + N` starting from `first` and proceeding to `last`. For each non-negative integer `n<N`, performs `*(result + n) = *(first + n)`.

In other words, the destination range *may overlap* with the source range, as long as the beginning of the destination is outside the source range. The typical use case is implementing a shift "to the left":

```
// Copy [3, 7) to the subrange [1, 5). The ranges overlap,
// but this behavior is well defined:

std::vector<int> v = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
//                       ^->   \----+----/
//                       |          |
//                       \----------/

std::copy(v.begin() + 3,
          v.begin() + 7,
          v.begin() + 1);

std::println("{}", v); // [0, 3, 4, 5, 6, 5, 6, 7, 8, 9]
```

The sequential nature of assignments (enshrined in the *Effects* clauses of these algorithms) makes the above work, even in the presence of an overlap. After the call to `copy`, the destination range contains the same values that were in the source range before the copy (we're assuming the type has "value semantics", and doesn't do anything strange in its assignment operators, such as mutating unrelated objects). In the example, the destination range indeed contains `[3, 4, 5, 6]`.

This makes an algorithm like `move` a natural building block for container erasure: the implementation of `erase` can move the tail of the container over the erased elements, then destroy the tail of moved-from elements:

```
template <typename T>
constexpr auto vector<T>::erase(iterator first, iterator last) -> iterator
{
    if (first == last)
        return last;

    auto new_end = std::move(last, end(), first);
    std::destroy(new_end, end());

    // ... update bookkeeping ...

    return first;
}
```

Vice versa, `copy_backward` and `move_backward` allow for the beginning of the destination range to overlap with the *end* of the source range (that is, shifting "to the right"), as needed by `vector::insert`.

§ 2.2. What is the problem of the existing preconditions?

§ 2.2.1. They are incomplete

Consider the following example, which satisfies the preconditions of `std::copy`:

```
// copy [3, 7) to [8, 5) (output is in reverse):

std::vector<int> v      = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
//                                  \----+----/  <-^
//                                       |         |
//                                       \---------/

std::copy(v.begin() + 3,
          v.begin() + 7,
          std::make_reverse_iterator(v.begin() + 9)); // starts at v[8], nc
```

The precondition of `copy` requires that the beginning of the destination range is not inside the source range. The destination starts at `v[8]`, which is outside the source range `[3, 7)`. The precondition is satisfied, and the behavior is therefore fully defined.

Yet, the output is "garbled": the output (reversed) range now contains `[3, 4, 5, 5]`, which *does not match the contents of the input range before the copy*:

```
std::println("{}", v); // [0, 1, 2, 3, 4, 5, 5, 4, 3, 9]
```

In short, the precondition fails to account for reverse iterators (or other iterators that may "jump" to arbitrary positions). As the example shows, a reverse iterator can satisfy the precondition (its initial position is outside the source range) while still "walking back" into the source range as it advances.

More fundamentally, this means that **the preconditions cannot guarantee a useful postcondition.** As shown before, a user may expect that `copy`, when called in contract, establishes as postcondition that the destination contains the same sequence that was in the source range before the copy. However, the reverse iterator example shows that this postcondition does not hold: the destination *does not* contain a faithful copy of the source range.

A precondition that fails to prevent the harm it claims to prevent is worse than no precondition at all: it creates a false sense of safety. Users who see no undefined behavior sanitizer warning may trust that their output is correct.

§ 2.2.2. They are too aggressive

A self-copy like `copy(first, last, first)` is formally undefined behavior. Although useless (or wasteful) in practice, this call should resolve in a number of self-assignments, which are supposed to be harmless. It is unclear therefore why the precondition is so "aggressive".

§ 2.2.3. They do not enable any useful optimization

The operations performed by these algorithms, including their order, are fully specified by the Standard: `copy` is specified to copy-assign elements from first to last, in that order; `copy_backward` goes from last to first; and so on (see [alg.copy]/3, [alg.copy]/28, etc.).

For non-contiguous iterators, the preconditions do not seem to enable any optimization that would not be possible without them: they merely document "what actually works" given a left-to-right order of assignments (or right-to-left for the backward algorithms).

For contiguous iterators over trivially copyable types, one might expect the precondition to allow an implementation to use `memcpy` (which is faster than `memmove` but does not handle overlapping ranges). However, this is not the case: a left overlap (destination starts before source) is explicitly legal, so `memcpy` cannot be used in general; implementations are therefore forced to use `memmove`. (Indeed, the major Standard Library implementations use `memmove` as a QOI optimization, and do not use `memcpy`.)

✧ ✧ ✧

In summary: the preconditions are both too strict (they forbid valid patterns that could work correctly, such as overlapping contiguous iterators) and too permissive (they allow patterns that produce garbled results).

3. Proposal

3.1. Remove the preconditions of the copy algorithms

We propose to amend the preconditions for `copy`, `move`, `copy_backward`, and `move_backward`. As argued above, the current preconditions offer no optimization benefit and cannot even guarantee a correct result.

Our goal is to remove this source of undefined behavior from the Standard Library.

We offer two options:

1. **Option A: "just remove" the preconditions.**

   With this option we would simply drop the *Preconditions* clauses, but we would not be proposing any change to the effects of the algorithms.

   Since the algorithms are specified to perform sequential assignments from first to last (or, for `copy_backward`/`move_backward`, last to first), this means that the result of an algorithm invocation is exactly what those assignments are going to produce; outcomes such as garbled output due to overlap, or reading from already-moved-from objects, etc., are simply the observable consequence of following the assignment order.

2. **Option B: remove the preconditions, but make the behavior erroneous if there is a self-clobbering read.**

Even in this case, we propose to remove the precondition, and keep the current specification (sequential assignments).

However: *if, during the execution of an algorithm, the algorithm reads from an element that has already been assigned to, we propose to mark this behavior as* ***erroneous***. The end result is still the fully-specified sequential-assignment result, but the program has a logical bug, that implementations are allowed to diagnose.

More precisely, we define a **self-clobbering read** as follows: during the execution of the algorithm, a self-clobbering read occurs when the algorithm reads the value of an element that has been already assigned-to by the same invocation of the algorithm.

This definition covers both the classical right-overlap case (destination starts inside the source range, which is right now undefined behavior) and the reverse-iterator case described in the Motivation section (which is right now well-defined behavior, but with a questionable outcome).

Note that:

- a self-copy (`copy(first, last, first)`) performs assignments where each destination element aliases the corresponding source element. These are not self-clobbering reads, because each element is read from the *same* position that was written to, with no intervening write to a *different* position that aliases the read position;
- the definition of self-clobbering read is only concerned about whether an assignment takes place, and the source of the assignment is an object that has already been assigned to by the algorithm itself. In other words, our definition is "positional": while the algorithm reads from positions 0, 1, 2, ..., *n*, we detect only whether the element at position `i` has already been assigned to by the algorithm. A more general notion of self-clobbering read may include the fact that, in principle, assigning to an element at position `i` may mutate an element at position `k` via side-effects. This more general notion is however pathological, and even hard to describe properly, so we are not attempting to model it.

Both options enshrine the assignment-order semantics already mandated by the Standard, as no implementation is taking advantage of the current precondition. The only difference is that Option A makes all such calls well-formed (with potentially wrong results), while Option B flags self-clobbering reads as erroneous behavior.

§ 3.2. The contiguous iterator extension

Under both Option A and Option B, the Standard would mandate a specific result for right-overlap: the sequential-assignment result (garbled values).

However, let's consider this right-overlap case:

```
// Copy [3, 7) to the subrange [5, 9):

std::vector<int> v = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
//                               \--+------/
//                                  |  ^->
//                                  +--/

std::copy(v.begin() + 3,
          v.begin() + 7,
          v.begin() + 5); // Currently: undefined behavior (precondition v
```

Now, if we were to adopt either option proposed above, the series of sequential assignments mandated by the specification of `std::copy` *should* produce:

```
std::vector<int> v = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

std::copy(v.begin() + 3,
          v.begin() + 7,
          v.begin() + 5); // Specified behavior (WD under Option A, EB unde

std::println("{}", v);
// Example is self-clobbering, expected output is:
// [0, 1, 2, 3, 4, 3, 4, 3, 4, 9]
//                ^^^^^^^^^^ garbled
```

However, this is not the result that existing implementations produce (again, this is formally undefined behavior at the moment). Existing implementations (Godbolt) in fact use `memmove` for contiguous ranges of trivially copyable types, which silently produces the *correct* result:

```
// [0, 1, 2, 3, 4, 3, 4, 5, 6, 9]
//                ^^^^^^^^^^ correct!
```

Under either option, this (pre-existing) behavior would be a violation of the specification, since both of them mandate the sequential-assignment (garbled) result. (Under Option B the program would have erroneous behavior, but still an implementation using `memmove` would deviate from the specification.)

We are concerned that *there may be existing code that is accidentally relying on this runtime behavior*; with the proposed change we would be introducing a regression in it.

In short: without an additional extension of the specification, both Option A and Option B would force implementations to pessimize. They would need to detect right-overlap with contiguous ranges of trivially copyable types, and fall back to element-by-element assignment in that case, and therefore they would need to give up the `memmove` optimization that implementations currently employ. This would be a performance regression with no benefit to correctness.

§ 3.2.1. Extension proposals for contiguous iterators

We therefore propose an optional extension (two options, actually) of the behavior for this family of algorithms: when the algorithms are used on contiguous iterators, the algorithms must detect if the source and destination ranges overlap, and *do the right thing*.

That is, they will always establish the postcondition that "the destination range equals the source range as it was before the copy" for all possible overlaps.

If paired with Option B, the extension would *lift the EB condition*: if the result is guaranteed to be correct, then there is no logical bug in the program, and its behavior should not be flagged as erroneous.

The only question is whether this extension should be limited to trivially copyable types only, or should instead be allowed for all types.

For the sake of exposition, let's enumerate the two possibilities:

1. **Extension E1: contiguous iterators over trivially copyable types only.**

   For contiguous iterators over trivially copyable types, mandate the correct postcondition for all overlaps.

2. **Extension E2: all contiguous iterator types.**

   Mandate the same "correct-copy" postcondition for *all* contiguous iterators. This is a strict superset of E1.

In both cases the idea is that implementations must detect the overlap direction and dispatch to a forward or backward copy accordingly. (In essence, `copy` dispatches to `copy_backward` internally when it detects a right overlap.) Since the existing algorithms already fully specify the order of assignments, we suggest to actually enshrine this implementation strategy in the effects (by stating something like that the order of assignments goes from last to first in case of right overlap).

✧ ✧ ✧

There are a few reasons for distinguishing the two cases.

As mentioned above, right now Standard Library implementations employ `memmove` (or similar) when copying/moving trivially copyable types, and `memmove` already yields the correct result; this means that these implementations will *not* have to change their "runtime" version of these algorithms. They would still however need to adapt their non-optimized ones, for instance the ones used during constant evaluation.

Therefore, **the extension E1 closely follows what implementations are already doing today;** it is a conservative choice.

On the other hand, **E2 extends the postcondition to non-trivially copyable types**. While the implementation challenge for E2 is going to be extremely similar to E1's, the main consequence of E2 for non-trivially copyable types is that the assignment order may differ from strict left-to-right: when a right overlap is detected, the implementation copies from right-to-left instead.

For non-trivially copyable types this assignment order is observable. We analyze the implications below.

§ 3.2.2. Implications of E2 for non-trivially copyable types

Extension E2 means that, when a right overlap is detected with contiguous iterators over non-trivially copyable types, the implementation will perform assignments from right to left instead of left to right. This has several observable consequences:

1. **Side effects in assignment operators change order.** A type with logging, reference counting, or other side effects in its copy or move assignment operator will observe these effects in a different order depending on the overlap direction. For instance, a logging assignment operator would log assignments in reverse order when right-overlap is detected.
2. **Exception safety characteristics change.** If the copy/move assignment operator throws an exception after K successful assignments, the identity of the K already-modified elements depends on the copy direction: left-to-right modifies the *first* K destination elements, while right-to-left modifies the *last* K.
3. **The effects clause needs amendment.** Currently, the effects of `copy` specify `*(result + n) = *(first + n)` for each `n` in increasing order. Under E2, the specification would need to state that assignments proceed from last to first when a right overlap is detected.

Despite these observable differences, **we believe E2 is the right choice**, for the following reasons:

- *No source compatibility issue*. Having a right overlap today is already undefined behavior; changing e.g. `std::copy` from doing left-to-right assignments to right-to-left assignments in this case is therefore not subject to any form of source compatibility.
- *E2 avoids a confusing asymmetry*. Under E1, trivially copyable types would silently produce the correct result, while non-trivially copyable types would get erroneous behavior and garbled output for the exact same overlap pattern. This distinction maps poorly onto user intent: users calling `std::copy` with overlapping contiguous ranges want the correct result regardless of the element type.
- *Implementation cost is negligible*. The overlap check is just a small constant number of pointer comparisons (after extracting addresses from the iterators). The dispatch to forward or backward copy is straightforward.

Apart from the technical aspects, we may also foresee concerns about teachability, as well as managing users' expectations: for instance, we'd have to teach that `std::copy` "normally" performs left-to-right assignments, except when the iterators are contiguous (and, in the E1 case, they're over trivially copyable types), in which case the algorithm does the smart thing and correctly handles possible overlaps, possibly by doing assignments right-to-left.

Of course, this can be fully justified by the fact that the algorithm tries its best to do the *correct thing*, but it can only be as smart as its inputs allow: contiguous iterators offer the information needed for it to be correct, and inferior iterator categories don't.

§ 3.3. Summary of the proposed changes

In this table we're summarizing the status quo and the changes brought by the options and extensions illustrated above. The table refers to `std::copy`, but similar changes are expected for the other algorithms of the family.

	Right-overlap, non-contiguous iterators	**Self-clobber satisfying the preconditions (e.g. reverse iterator)**	**Right-overlap, contiguous + trivially copyable**	**Right-overlap, contiguous + non-trivially copyable**
Current	UB	WD, garbled	UB [1]	UB
A + no extension	WD, garbled	WD, garbled	WD, garbled [2]	WD, garbled

	Right-overlap, non-contiguous iterators	Self-clobber satisfying the preconditions (e.g. reverse iterator)	Right-overlap, contiguous + trivially copyable	Right-overlap, contiguous + non-trivially copyable
A + E1	WD, garbled	WD, garbled	WD, correct	WD, garbled
A + E2	WD, garbled	WD, garbled	WD, correct	WD, correct
B + no extension	EB, garbled	EB, garbled [3]	EB, garbled [2]	EB, garbled
B + E1	EB, garbled	EB, garbled [3]	WD, correct	EB, garbled
B + E2 (recommended)	EB, garbled	EB, garbled [3]	WD, correct	WD, correct

[1] `memmove` in practice, yielding the "correct" result

[2] Existing implementations using `memmove` for contiguous ranges of trivially copyable types would deviate from the specification: `memmove` produces the correct result, whereas under these options we would mandate the garbled sequential-assignment result.

[3] For this case, Option B would make things *more strict* than the current behavior, by flagging clobber cases (currently well-defined) as erroneous behavior.

§ 3.4. Suggested polls and recommendations

We would like to poll the various options presented above.

Technically speaking, Option A and Option B can be picked completely independently from the extensions (none, E1, or E2). Choosing neither extension is technically valid, but forces implementations to change (and to pessimize).

Since a change is anyhow required, **we recommend Option B + Extension E2**; that is, to:

- **remove all undefined behavior** by dropping the preconditions;
- **introduce erroneous behavior for self-clobbering reads** (on non-contiguous iterators);
- and **guarantee the correct-copy postcondition (with no EB) for all contiguous iterators**. This a pure extension, and requires specifying these new effects.

§ 4. Design decisions

§ 4.1. What about `copy_n`'s preconditions?

`copy_n` currently has no preconditions, which is Library Defect [LWG3089]. There is little reason for it to differ from `copy`; **we therefore propose to amend `copy_n`'s specification** so that it has the same preconditions and behavior (incl. the conditions for erroneous behavior) as `copy` after this paper.

§ 4.2. Parallel versions are out of scope

The parallel overloads of `copy` and `move` require that the source and destination ranges do not overlap at all, and we do not propose to change this. The reason is sound: overlapping ranges in a parallel context might introduce data races which may be expensive to avoid (**if** they can be avoided at all).

However, for the parallel overloads of `copy_n`, we propose to **establish** a new precondition; that is, that the source and destination ranges do not overlap. This matches the specification of the parallel overload of `copy`.

§ 4.3. Specialized memory algorithms are out of scope

The specialized memory algorithms (`uninitialized_copy`, etc.) are not affected by our proposal.

These algorithms construct objects into uninitialized storage, rather than assigning onto existing objects. Since in general an object cannot be constructed into storage that already holds a live object, the possibility of overlap between source and destination does not arise.

§ 5. Impact on the Standard

This proposal relaxes the preconditions on certain algorithms in the Standard Library (`copy`, `copy_n`, `move`, `copy_backward`, and `move_backward`). For `copy_n`, it establishes new preconditions, matching existing practice and resolving [LWG3089].

This paper does not add any new entity and it does not affect the core language. It has no ABI implications.

Code that exhibited undefined behavior becomes either well-formed (Option A) or erroneous behavior (Option B).

Under Option B, code that was well-formed but had "questionable" behavior (garbled output) becomes erroneous.

With the proposed extensions, we also make these algorithms always establish a useful postcondition in any possible overlap, when they work over contiguous iterators.

For `constexpr` evaluation: the `constexpr` code paths of these algorithms may need to be amended to detect overlap direction and choose the correct assignment order (under the extensions), but this is straightforward.

§ 6. Future work

The reasoning applied here to `copy`, `move`, `copy_backward`, and `move_backward` could be extended to other non-parallel algorithms in the copy family (e.g., `copy_if`, `replace_copy`, `remove_copy`).

They could be also extended to the relocation algorithms proposed by [P3516R2]. The considerations for the specialized memory algorithms discussed above do not fully apply to those algorithms, as they create *and* destroy objects while relocating.

The direction in LEWG seems to be toward banning all overlaps for these algorithms. We could instead relax their preconditions and once again express them as erroneous behavior: the algorithm does the expected thing (in-order assignments), a logical bug is flagged, and a source of UB is removed.

For most of these other algorithms the contiguous iterator extension does not apply, as the output size is not necessarily known in advance.

§ 7. Proposed Wording

All changes are relative to [N5032].

We are going to provide wording that applies the recommended choice of Option B and Extension E2.

§ 7.1. Expressing self-clobbering reads in the wording

In the wording we are not formally introducing the notion of self-clobbering read. Since our definition above is positional, we limit ourselves to a *position-based overlap detection* (in the wording, *overlapping*), whose purpose is to detect a specific situation: given the algorithm's natural iteration order, will

a later read access an object that an earlier write has already modified? Note that this is a more strict condition than a mere overlap of the source and destination ranges.

For forward algorithms (`copy`, `copy_n`, `move`), the natural iteration order goes from $n = 0$ upward to N. The n-th step reads from `*(first + n)` and writes into `*(result + n)`. A self-clobbering read occurs here when a step n reads from an object that a prior step m (that is, with $m < n$) wrote to; that is, it occurs when `*(first + n)` and `*(result + m)` refer to the same object. In other words, if such m and n exist (with $m < n$), then the input and output ranges overlap in a way that the algorithms will perform a self-clobbering read.

We can show that this correctly model the algorithms:

- For the **left-overlap** case, consider `copy` with `first = v.begin() + 3`, `last = v.begin() + 7`, and `result = v.begin() + 1`. Here `*(first + n)` corresponds to element $3+n$; and `*(result + m)` corresponds to element $1+m$. They refer to the same object when $3+n = 1+m$, which implies $m = n + 2$. But then $m < n$ requires $n + 2 < n$, which is impossible. Therefore, there is no self-clobbering read here, and this case still has well-defined behavior.
- For the **right-overlap** case, consider `copy` with `first = v.begin() + 3`, `last = v.begin() + 7`, and `result = v.begin() + 5`. Here `*(first + n)` corresponds to element $3+n$; and `*(result + m)` corresponds to element $5+m$. They refer to the same object when $3+n = 5+m$, that is, $n = m + 2$. The condition $m < n$ becomes $m < m + 2$, which is always true. Therefore, the condition detects the self-clobbering read.
- For the **reverse iterator** case, recalling the motivating example, consider `copy` with `first = v.begin() + 3`, `last = v.begin() + 7`, and `result = make_reverse_iterator(v.begin() + 9)`. Here `*(first + n)` corresponds to element $3+n$; and `*(result + m)` corresponds to element $8-m$. These are the same object when $3+n = 8-m$, that is, $m+n = 5$. With $m = 2$ and $n = 3$ the $m < n$ condition is satisfied (indeed, the object at `v[6]` will be read after it's been written into).
- For the **self-copy** case, when `result = first`, then `*(first + n)` will correspond to the same object as `*(result + m)` only when $m = n$, which does not satisfy $m < n$. The condition is not satisfied; the algorithm has well-defined behavior and performs self-assignments.

For the backward algorithms (`copy_backward` and `move_backward`) the analysis is symmetric. The natural order for these algorithms is from $n = 1$ upward, where step n reads from `*(last - n)` and writes into `*(result - n)`. The problematic case for these algorithms is left-overlapping ranges (destination starts before source), where a later step's read aliases an earlier step's write.

For instance, consider `copy_backward` with `first = v.begin() + 3`, `last = v.begin() + 7`, and `result = v.begin() + 5` (where step $n = 1$ writes into `v[4]` and step $n = 3$ reads from it).

The condition `*(last - n)` and `*(result - m)` referring to the same object with $m < n$ correctly detects this with $m = 1, n = 3$.

§ 7.2. Feature-testing macro

In [version.syn], add a new feature-testing macro:

```
#define __cpp_lib_copy_relaxed_overlap YYYYMML // freestanding, also in <a
```

For brevity's sake, we are proposing one macro for the entire family of functions (`copy`, `copy_n`, `copy_backward`, `move`, `move_backward`).

§ 7.3. copy

Modify [alg.copy]/1-5 as shown:

```
template<class InputIterator, class OutputIterator>
  constexpr OutputIterator copy(InputIterator first, InputIterator last,
                                OutputIterator result);

template<input_iterator I, sentinel_for<I> S, weakly_incrementable O>
  requires indirectly_copyable<I, O>
  constexpr ranges::copy_result<I, O> ranges::copy(I first, S last, O res
template<input_range R, weakly_incrementable O>
  requires indirectly_copyable<iterator_t<R>, O>
  constexpr ranges::copy_result<borrowed_iterator_t<R>, O> ranges::copy(R
```

- Let *N* be `last - first`.
- Let `InIter` be `InputIterator` for the overload in namespace `std`, `I` for the first overload in namespace `ranges`, and `iterator_t<R>` for the second overload in namespace `ranges`.
- Let `OutIter` be `OutputIterator` for the overload in namespace `std`, and `O` for the overloads in namespace `ranges`.
- Let *overlapping* be `true` if there exist non-negative integers *m* and *n* with $m < n < N$ such that `*(first + n)` and `*(result + m)` refer to the same object; otherwise `false`.
- ~~*Preconditions*: `result` is not in the range `[first, last)`.~~
- *Effects*: ~~Copies elements in the range `[first, last)` into the range `[result, result + N)` starting from `first` and proceeding to `last`. For each non-negative integer $n < N$, performs `*(result + n) = *(first + n)`.~~
  - If *N* equals `0`, no effects.
  - Otherwise, if `InIter` and `OutIter` both model `contiguous_iterator` and *overlapping* is `true`, for each non-negative integer $n < N$, performs `*(result + n) = *(first + n)`, starting from $n = N - 1$ and proceeding in decreasing order.
  - Otherwise, for each non-negative integer $n < N$, performs `*(result + n) = *(first + n)`, starting from $n = 0$ and proceeding in increasing order. If *overlapping* is `true`, the behavior is erroneous.
- *Returns*:
  - `result + N` for the overload in namespace `std`.

    - `{last, result + N}` for the overloads in namespace `ranges`.
- *Complexity*: Exactly *N* assignments.

7.4. copy_n

For `copy_n` ([alg.copy]/12-18) we are going to split the specification in the sequential overloads (to which we are going to apply the changes suggested by this paper) and parallel overloads. The latter are going to follow the parallel `copy` overloads, including adding a non-overlap precondition which is currently missing.

Duplicate the existing wording for [alg.copy]/12-18; apply these changes to the *first* copy:

```
template<class InputIterator, class Size, class OutputIterator>
  constexpr OutputIterator copy_n(InputIterator first, Size n,
                                  OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class Size, class
  ForwardIterator2 copy_n(ExecutionPolicy&& exec,
                          ForwardIterator1 first, Size n,
                          ForwardIterator2 result);

template<input_iterator I, weakly_incrementable O>
  requires indirectly_copyable<I, O>
  constexpr ranges::copy_n_result<I, O>
    ranges::copy_n(I first, iter_difference_t<I> n, O result);

template<execution_policy Ep, random_access_iterator I, random_access_ite
         sized_sentinel_for<O> OutS>
  requires indirectly_copyable<I, O>
  ranges::copy_n_result<I, O>
    ranges::copy_n(Ep&& exec, I first, iter_difference_t<I> n, O result,
```

- Let ~~*M*~~ *N* be max(0, `n`).
- ~~Let `result_last` be `result + M` for the overloads with no parameter `result_last`.~~
- ~~Let *N* be `min(result_last - result, M)`.~~
- Let `InIter` be `InputIterator` for the overload in namespace `std`, and `I` for the overload in namespace `ranges`. Let `OutIter` be `OutputIterator` for the overload in namespace `std`, and `O` for the overload in namespace `ranges`.
- Let *overlapping* be `true` if there exist non-negative integers *m* and *i* with $m < i < N$ such that `*(first + i)` and `*(result + m)` refer to the same object; otherwise `false`.
- *Mandates*: The type `Size` is convertible to an integral type ([conv.integral], [class.conv]).
- *Effects*: ~~For each non-negative integer $i < N$, performs `*(result + i) = *(first + i)`.~~
  - If `N` equals 0, no effects.
  - Otherwise, if `InIter` and `OutIter` both model `contiguous_iterator` and *overlapping* is `true`, for each non-negative integer $i < N$, performs `*(result + i) = *(first + i)`, starting from $i = N - 1$ and proceeding in decreasing order.

- Otherwise, for each non-negative integer $i < N$, performs `*(result + i) = *(first + i)`, starting from $i = 0$ and proceeding in increasing order. If *overlapping* is `true`, the behavior is erroneous.

- *Returns*:
  - `result + N` for the overload in namespace `std`.
  - `{first + N, result + N}` for the overload in namespace `ranges`.
- *Complexity*: Exactly N assignments.

Apply these changes to the *second* copy:

```
template<class InputIterator, class Size, class OutputIterator>
  constexpr OutputIterator copy_n(InputIterator first, Size n,
                                  OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class Size, class
  ForwardIterator2 copy_n(ExecutionPolicy&& exec,
                          ForwardIterator1 first, Size n,
                          ForwardIterator2 result);

template<input_iterator I, weakly_incrementable O>
  requires indirectly_copyable<I, O>
  constexpr ranges::copy_n_result<I, O>
    ranges::copy_n(I first, iter_difference_t<I> n, O result);

template<execution-policy Ep, random_access_iterator I, random_access_ite
         sized_sentinel_for<O> OutS>
  requires indirectly_copyable<I, O>
  ranges::copy_n_result<I, O>
    ranges::copy_n(Ep&& exec, I first, iter_difference_t<I> n, O result,
```

- Let *M* be max(0, `n`).
- Let `result_last` be `result + M` for the ~~overloads with no parameter `result_last`~~ over-load in namespace `std` .
- Let *N* be min(`result_last - result`, *M*).
- *Mandates*: The type `Size` is convertible to an integral type ([conv.integral], [class.conv]).
- *Preconditions*: The ranges `[first, first + N)` and `[result, result + N)` do not overlap.
- *Effects*: For each non-negative integer $i < N$, performs `*(result + i) = *(first + i)`.
- *Returns*:
  - `result + N` for the overload in namespace `std`.
  - `{first + N, result + N}` for the overload in namespace `ranges`.
- *Complexity*: Exactly *N* assignments.

§ 7.5. copy_backward

Modify [alg.copy]/26-30 as shown:

```
template<class BidirectionalIterator1, class BidirectionalIterator2>
  constexpr BidirectionalIterator2
    copy_backward(BidirectionalIterator1 first,
                  BidirectionalIterator1 last,
                  BidirectionalIterator2 result);

template<bidirectional_iterator I1, sentinel_for<I1> S1, bidirectional_it
  requires indirectly_copyable<I1, I2>
  constexpr ranges::copy_backward_result<I1, I2>
    ranges::copy_backward(I1 first, S1 last, I2 result);
template<bidirectional_range R, bidirectional_iterator I>
  requires indirectly_copyable<iterator_t<R>, I>
  constexpr ranges::copy_backward_result<borrowed_iterator_t<R>, I>
    ranges::copy_backward(R&& r, I result);
```

- Let *N* be `last - first`.
- Let `InIter` be `BidirectionalIterator1` for the overload in namespace `std`, `I1` for the first overload in namespace `ranges`, and `iterator_t<R>` for the second overload in namespace `ranges`.
- Let `OutIter` be `BidirectionalIterator2` for the overload in namespace `std`, `I2` for the first overload in namespace `ranges`, and `I` for the second overload in namespace `ranges`.
- Let *overlapping* be `true` if there exist positive integers *m* and *n* with $m < n < N$ such that `*(last - n)` and `*(result - m)` refer to the same object; otherwise `false`.
- ~~*Preconditions*: `result` is not in the range `(first, last]`.~~
- *Effects*: ~~Copies elements in the range `[first, last)` into the range `[result, result + N)` starting from `first` and proceeding to `last`. For each non-negative integer $n < N$, performs `*(result - n) = *(last - n)`.~~
  - If *N* equals `0`, no effects.
  - Otherwise, if `InIter` and `OutIter` both model `contiguous_iterator` and *overlapping* is `true`, for each positive integer $n \le N$, performs `*(result - n) = *(last - n)`, starting from $n = N$ and proceeding in decreasing order.
  - Otherwise, for each positive integer $n \le N$, performs `*(result - n) = *(last - n)`, starting from $n = 1$ and proceeding in increasing order. If *overlapping* is `true`, the

behavior is erroneous.†

- *Returns*:
  - `result + N` for the overload in namespace `std`.
  - `{last, result + N}` for the overloads in namespace `ranges`.
- *Complexity*: Exactly *N* assignments.

† ~~`copy_backward` can be used instead of `copy` when `last` is in the range `[result - N, result)`.~~ When both iterators are contiguous, `copy` and `copy_backward` handle all overlaps correctly. For non-contiguous iterators, `copy_backward` can be used instead of `copy` when `last` is in the range `[result - N, result)`; using `copy` in that case has erroneous behavior.

§ 7.6. move

Modify [alg.move]/1-5 as shown:

```
template<class InputIterator, class OutputIterator>
  constexpr OutputIterator move(InputIterator first, InputIterator last,
                                OutputIterator result);

template<input_iterator I, sentinel_for<I> S, weakly_incrementable O>
  requires indirectly_movable<I, O>
  constexpr ranges::move_result<I, O>
    ranges::move(I first, S last, O result);
template<input_range R, weakly_incrementable O>
  requires indirectly_movable<iterator_t<R>, O>
  constexpr ranges::move_result<borrowed_iterator_t<R>, O>
    ranges::move(R&& r, O result);
```

- Let *E(n)* be
  - `std::move(*(first + n))` for the overload in namespace `std`;
  - `ranges::iter_move(first + n)` for the overloads in namespace `ranges`.

  Let *N* be `last - first`.
- Let `InIter` be `InputIterator` for the overload in namespace `std`, `I` for the first overload in namespace `ranges`, and `iterator_t<R>` for the second overload in namespace `ranges`.
- Let `OutIter` be `OutputIterator` for the overload in namespace `std`, and `O` for the overloads in namespace `ranges`.
- Let *overlapping* be `true` if there exist non-negative integers *m* and *n* with *m* < *n* < *N* such that `*(first + n)` and `*(result + m)` refer to the same object; otherwise `false`.
- ~~*Preconditions*: `result` is not in the range `[first, last)`.~~
- *Effects*: ~~Moves elements in the range `[first, last)` into the range `[result, result + N)` starting from `first` and proceeding to `last`. For each non-negative integer *n* < *N*, performs `*(result + n) = E(n)`.~~
  - If *N* equals `0`, no effects.
  - Otherwise, if `InIter` and `OutIter` both model `contiguous_iterator` and *overlapping* is `true`, for each non-negative integer *n* < *N*, performs `*(result + n) = E(n)`, starting from *n* = *N* - 1 and proceeding in decreasing order.

- Otherwise, for each non-negative integer $n < N$, performs `*(result + n) = E(n)`, starting from $n = 0$ and proceeding in increasing order. If *overlapping* is `true`, the behavior is erroneous.

- *Returns*:
  - `result + N` for the overload in namespace `std`.
  - `{last, result + N}` for the overloads in namespace `ranges`.
- *Complexity*: Exactly N assignments.

§ 7.7. move_backward

Modify [alg.move]/13-17 as shown:

```
template<class BidirectionalIterator1, class BidirectionalIterator2>
  constexpr BidirectionalIterator2
    move_backward(BidirectionalIterator1 first, BidirectionalIterator1 la
                  BidirectionalIterator2 result);

template<bidirectional_iterator I1, sentinel_for<I1> S1, bidirectional_it
  requires indirectly_movable<I1, I2>
  constexpr ranges::move_backward_result<I1, I2>
    ranges::move_backward(I1 first, S1 last, I2 result);
template<bidirectional_range R, bidirectional_iterator I>
  requires indirectly_movable<iterator_t<R>, I>
  constexpr ranges::move_backward_result<borrowed_iterator_t<R>, I>
    ranges::move_backward(R&& r, I result);
```

- Let *E(n)* be
  - `std::move(*(last - n))` for the overload in namespace `std`;
  - `ranges::iter_move(last - n)` for the overloads in namespace `ranges`.

  Let *N* be `last - first`.
- Let `InIter` be `BidirectionalIterator1` for the overload in namespace `std`, `I1` for the first overload in namespace `ranges`, and `iterator_t<R>` for the second overload in namespace `ranges`.
- Let `OutIter` be `BidirectionalIterator2` for the overload in namespace `std`, `I2` for the first overload in namespace `ranges`, and `I` for the second overload in namespace `ranges`.
- Let *overlapping* be `true` if there exist positive integers *m* and *n* with $m < n < N$ such that `*(last - n)` and `*(result - m)` refer to the same object; otherwise `false`.
- ~~*Preconditions*: `result` is not in the range `(first, last]`.~~
- *Effects*: ~~Moves elements in the range `[first, last)` into the range `[result, result + N)` starting from `first` and proceeding to `last`. For each non-negative integer $n < N$, performs `*(result - n) = E(n)`.~~
  - If *N* equals `0`, no effects.

- Otherwise, if `InIter` and `OutIter` both model `contiguous_iterator` and *overlapping* is `true`, for each positive integer $n \le N$, performs `*(result - n) = E(n)`, starting from $n = N$ and proceeding in decreasing order.
- Otherwise, for each positive integer $n \le N$, performs `*(result - n) = E(n)`, starting from $n = 1$ and proceeding in increasing order. If *overlapping* is `true`, the behavior is erroneous. †

- *Returns*:
  - `result - N` for the overload in namespace `std`.
  - `{last, result - N}` for the overloads in namespace `ranges`.
- *Complexity*: Exactly N assignments.

† ~~`move_backward` can be used instead of `move` when `last` is in the range `[result - N, result)`.~~ When both iterators are contiguous, `move` and `move_backward` handle all overlaps correctly. For non-contiguous iterators, `move_backward` can be used instead of `move` when `last` is in the range `[result - N, result)`; using `move` in that case has erroneous behavior.

8. Acknowledgements

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

References

Non-Normative References

[LWG3089]
Marshall Clow. *copy_n should require non-overlapping ranges*. New. URL: https://wg21.link/lwg3089

[N5032]
Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 15 December 2025. URL: https://wg21.link/n5032

[P3179R8]
Ruslan Arutyunyan, Alexey Kukanov, Bryce Adelstein Lelbach. *C++ parallel range algorithms*. 18 May 2025. URL: https://wg21.link/p3179r8

[P3516R2]

Louis Dionne, Giuseppe D'Angelo. *Uninitialized algorithms for relocation*. 16 May 2025. URL: https://wg21.link/p3516r2